

Car Servicing Company Database System

Entity-Relationship Diagram — Database Schema — Limitations & Assumptions

Entity-Relationship Diagram

ER Diagram

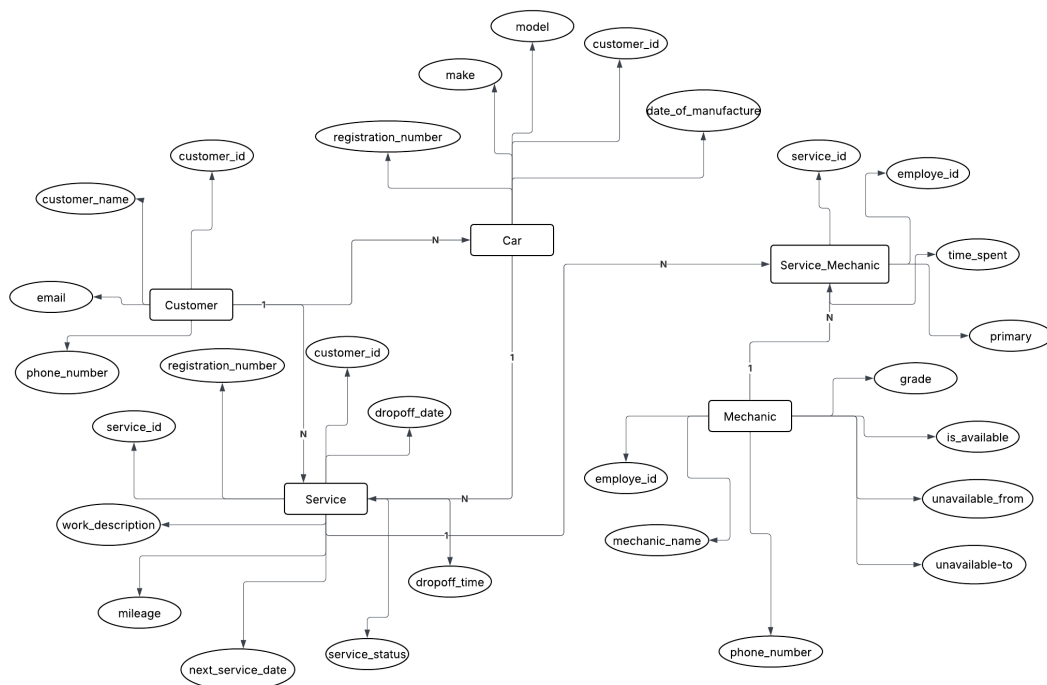


Figure 1: Entity Relationship Diagram

Cardinality Ratios Justification

Relationship	Cardinality	Justification
CUSTOMER to CAR	1:N	A single client may possess more than one vehicle. There is one owner for each car.
CAR to SERVICE	1:N	One car can have numerous services. Each service is for exactly one car.
CUSTOMER to SERVICE	1:N	A single customer can book several services. Each service is booked by exactly one customer.
SERVICE to MECHANIC	M:N	There can be multiple mechanics per service. Each mechanic can perform various services.

Participation Constraints Justification

- **CUSTOMER participation in owns:** *Partial (0..N)* - Customers are not required to own cars initially.
- **CAR participation in owned by:** *Total (1..1)* - Each car must have an owner.
- **CAR participation in is serviced:** *Partial (0..N)* - New cars may have no service history.
- **SERVICE participation in is for:** *Total (1..1)* - All services must be for a car.
- **SERVICE participation in booked by:** *Total (1..1)* - Each service must be customer-based.
- **SERVICE participation in executed by:** *Partial to Total* - Mechanics can be assigned later.
- **MECHANIC participation in performs:** *Partial (0..N)* - New mechanics can have no assignments.

Database Schema

Relational Schema (3NF Compliant)

Table	Schema	Constraints
CUSTOMER	(<u>customer_id</u> , customer_name, email, phone_number)	PK: customer_id UNIQUE: email
CAR	(<u>registration_number</u> , make, model, manufacture_date, customer_id)	PK: registration_number FK: customer_id → CUSTOMER
MECHANIC	(<u>employee_id</u> , mechanic_name, grade, phone_number, is_available, unavailable_by, unavailable_at)	PK: employee_id CHECK: grade IN (<i>list of values</i>)
SERVICE	(<u>service_id</u> , customer_id, registration_number, dropoff_date, dropoff_time, work_description, mileage, next_service_date, service_status)	PK: service_id FK1: customer_id → CUSTOMER FK2: registration_number → CAR
SERVICE_MECHANIC	(<u>service_id, employee_id</u> , time_spent)	PK: Composite (service_id, employee_id) FK1: service_id → SERVICE FK2: employee_id → MECHANIC

Data Types and Justifications

Attribute	Data Type	Justification
customer_id, employee_id, service_id	INT AUTO_INCREMENT	Efficient indexing, unique ID generation.
customer_name, mechanic_name	VARCHAR(100)	Accommodates full names with titles.
email	VARCHAR(100) UNIQUE	Standard email length, prevents duplication.
phone_number	VARCHAR(20)	International numbers with formatting characters.
registration_number	VARCHAR(20)	Natural key, supports UK plates and international formats.
make, model	VARCHAR(50)	Adequate for manufacturer names.
manufacture_date, dropoff_date	DATE	Date component only when time is not needed.
dropoff_time	TIME	Precise time recording.
work_description	TEXT	Unlimited length for detailed descriptions.
mileage	INT	Integer, must be greater than or equal to 0.
grade	VARCHAR(20)	Skill levels, with CHECK constraints.
time_spent	DECIMAL(5,2)	Hours with 2 decimal precision (max 999.99).
service_status	VARCHAR(20)	Status tracking with CHECK constraints.
is_available	BOOLEAN	Simple availability flag.
unavailable_by, unavailable_at	DATE NULL	Optional period tracking.

Normalization to 3NF

Normal Form	Compliance	Explanation
1NF	Achieved	Atomic values, no repeating groups, primary keys defined.
2NF	Achieved	No partial dependencies; e.g., <code>time_spent</code> depends on the full composite key in <code>SERVICE_MECHANIC</code> .
3NF	Achieved	No transitive dependencies; all non-key attributes depend directly on the primary key.

Junction Table Justification: The `SERVICE_MECHANIC` table resolves the M:N relationship between `SERVICE` and `MECHANIC` while storing relationship-specific attributes (`time_spent`).

Limitations and Assumptions

Design Assumptions

Assumption	Rationale
Customers registered before car registration.	Referential integrity is guaranteed for ownership tracking.
Unique email addresses per customer.	Prevents duplicate email usage for communication.
Registration number as natural primary key.	UK plates are uniquely used to identify vehicles permanently.
Defined service status workflow (e.g., Pending → In Progress → Completed).	Tracks service life cycle systematically.
Services can be created without mechanic assignment initially.	Supports the business need for flexible booking.
Time spent is recorded in hours with 2 decimal places.	Common practice which permits quarter-hour increments.
Single drop-off date per service.	Matches typical booking patterns, streamlines scheduling.
Next service date is stored after completion.	Enables proactive customer reminders.

Design Limitations

Limitation	Impact	Potential Mitigation
No service completion date.	Cannot calculate service duration or generate completion reports.	Add <code>completion.date</code> and <code>pickup.date</code> fields.
Limited mechanic unavailability tracking.	Cannot handle multiple separate vacation periods.	Create a separate <code>MECHANIC.UNAVAILABILITY</code> table.
No parts or pricing information.	Cannot generate invoices or track inventory.	Add <code>PARTS</code> and <code>SERVICE.PARTS</code> tables.
No service history comparison.	Limited ability to produce maintenance reminders.	Develop views or stored procedures for analysis.
No appointment scheduling.	Cannot prevent double-booking or optimize schedules.	Add an <code>APPOINTMENT.SLOT</code> table.
No communication history tracking.	Cannot track customer reminders or correspondence.	Add a <code>COMMUNICATION.LOG</code> table.
No service bay/location tracking.	Cannot optimize workspace allocation.	Add a <code>SERVICE.BAY</code> table.
Limited concurrent update handling.	Potential for conflicting mechanic assignments.	Implement application-level locking or transactions.
No audit trail.	Unable to trace who modified data or when.	Add audit tables with triggers.
Single customer per service.	Cannot handle shared-cost scenarios.	Add a <code>CUSTOMER.SERVICE</code> junction table.